

Contents

1	Anthropic End-to-End Flow Diagram	1
1.1	End-to-End Flow Overview	1
1.2	Flow Phases	1
1.2.1	1. User Request Phase	1
1.2.2	2. Configuration Phase	3
1.2.3	3. Auth Resolution Phase	3
1.2.4	4. Session Setup Phase	3
1.2.5	5. Stream Function Selection	4
1.2.6	6. Agent Execution Phase	4
1.2.7	7. Streaming Response Phase	5
1.2.8	8. Completion Phase	6
1.2.9	9. Error Handling	6
1.3	Key Flow Characteristics	7
1.3.1	Simplicity vs Azure OpenAI	7
1.3.2	Advantages	7
1.3.3	Trade-offs	8
1.4	Tool Use Flow	8
1.5	Performance Optimizations	8
1.5.1	1. Prompt Caching	8
1.5.2	2. Streaming	8
1.5.3	3. Session Caching	8
1.6	Comparison: Full Request Lifecycle	9
1.6.1	Azure OpenAI (Managed Identity)	9
1.6.2	Anthropic (API Key/Token)	9
1.7	Configuration Examples	9
1.7.1	Provider Config	9
1.7.2	Auth Profile (Token Mode)	9
1.7.3	Environment Variables	10
1.8	Summary	10

1 Anthropic End-to-End Flow Diagram

1.1 End-to-End Flow Overview

This diagram shows the **complete request/response cycle** for Anthropic models in clawdbot, from user message to API response.

1.2 Flow Phases

1.2.1 1. User Request Phase

Components: User → Gateway/Channel → Runner

1. User sends message via channel (WhatsApp, Telegram, Discord, etc.)
2. Gateway receives and parses message
3. Gateway extracts context (conversation history, session info)

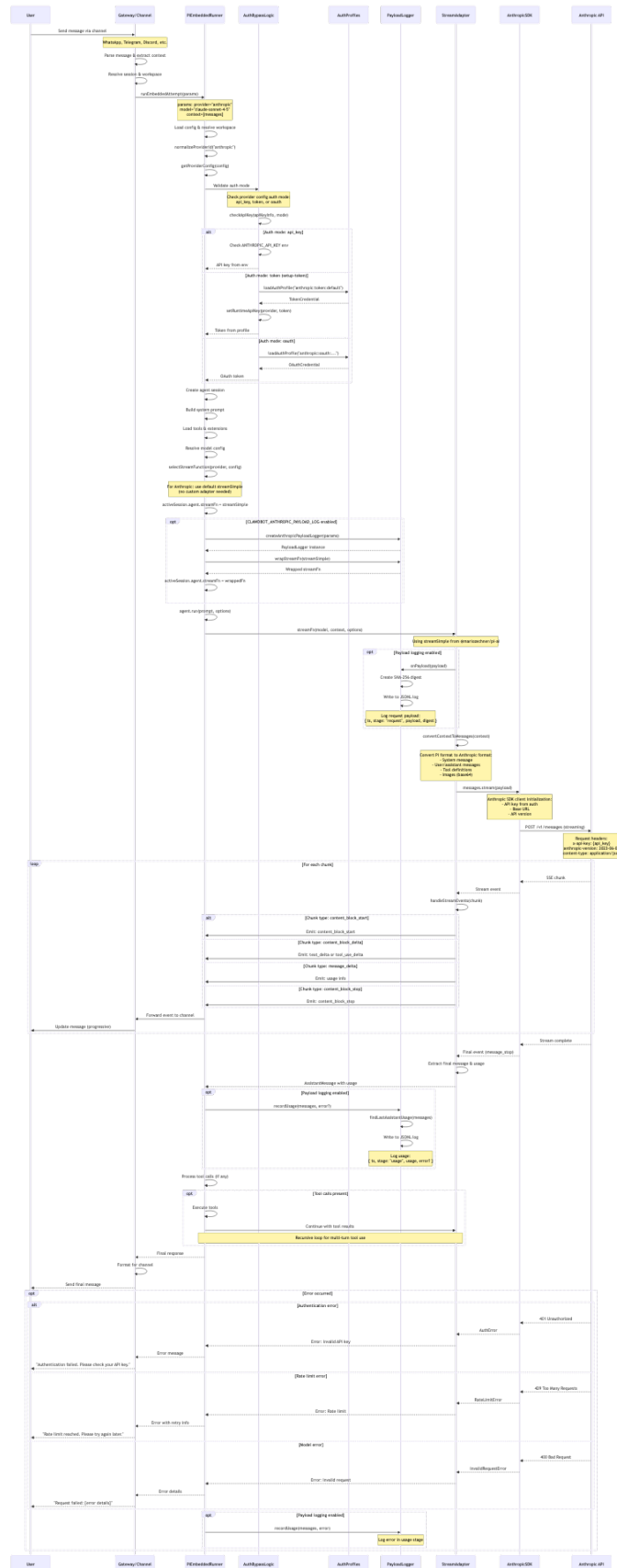


Figure 1: Diagram 1

4. Gateway resolves session and workspace directory
 5. Gateway calls `runEmbeddedAttempt()` with parameters
-

1.2.2 2. Configuration Phase

Components: Runner (internal)

1. Load configuration from workspace and global settings
 2. Normalize provider ID: "anthropic" → "anthropic"
 3. Get provider config from `config.models.providers.anthropic`
 4. Extract auth mode, base URL, and API version
-

1.2.3 3. Auth Resolution Phase

Components: Runner → AuthBypassLogic → AuthProfiles

Three auth modes supported:

1.2.3.1 A. API Key Mode (`api_key`)

1. Check `ANTHROPIC_API_KEY` environment variable
2. If found, use directly
3. If not found, `throw error` (not configured)

1.2.3.2 B. Token Mode (`token - setup-token`)

1. Load auth profile: `"anthropic:token:default"` (or named profile)
2. Extract token `from` credential
3. `Set` runtime API key `in` auth storage
4. Use token `in` API requests

1.2.3.3 C. OAuth Mode (`oauth`)

1. Load OAuth profile `from` auth profiles
 2. Extract OAuth `credentials` (access token, refresh token)
 3. Use OAuth token `in` API requests
-

1.2.4 4. Session Setup Phase

Components: Runner (internal)

1. Create agent session using Pi AI framework
 2. Build system prompt from templates and workspace files
 3. Load tools (file operations, bash, web search, etc.)
 4. Load extensions and skill definitions
 5. Resolve model configuration (max tokens, temperature, etc.)
-

1.2.5 5. Stream Function Selection

Components: Runner → StreamAdapter

For Anthropic:

```
// Use default streamSimple (no custom adapter)
activeSession.agent.streamFn = streamSimple

// Optional: Wrap with payload logger
if (CLAWBOT_ANTHROPIC_PAYLOAD_LOG === 'true') {
  const logger = createAnthropicPayloadLogger(params)
  activeSession.agent.streamFn = logger.wrapStreamFn(streamSimple)
}
```

Comparison with Azure OpenAI: - Azure OpenAI: Uses custom `streamAzureOpenAINative` adapter - Anthropic: Uses default `streamSimple` from Pi AI

1.2.6 6. Agent Execution Phase

Components: Runner → StreamAdapter → AnthropicSDK → API

1.2.6.1 6a. Context Conversion

```
// Convert Pi format to Anthropic format
{
  system: "You are a helpful assistant...",
  messages: [
    { role: "user", content: "Hello" },
    { role: "assistant", content: "Hi! How can I help?" }
  ],
  tools: [...],
  max_tokens: 4096,
  temperature: 0.7
}
```

1.2.6.2 6b. API Request

POST <https://api.anthropic.com/v1/messages>

Headers:

```
x-api-key: {api_key_or_token}
anthropic-version: 2023-06-01
content-type: application/json
```

Body:

```
{
  "model": "claude-sonnet-4-5-20250929",
  "system": "...",
  "messages": [...],
}
```

```

    "stream": true,
    "max_tokens": 4096
}

```

1.2.6.3 6c. Optional: Log Request

If payload logging is enabled:

```

{
  "ts": "2026-02-09T20:00:00.000Z",
  "stage": "request",
  "runId": "run_123",
  "sessionId": "session_456",
  "provider": "anthropic",
  "modelId": "claude-sonnet-4-5-20250929",
  "modelApi": "anthropic-messages",
  "workspaceDir": "/Users/user/project",
  "payload": { ... },
  "payloadDigest": "abc123..."
}

```

1.2.7 7. Streaming Response Phase

Components: API → SDK → StreamAdapter → Runner → Gateway → User

Server-Sent Events (SSE) chunks:

1.2.7.1 Chunk Types:

1. **message__start:** Initial metadata

```

{ "type": "message_start", "message": { "id": "...", "role": "assistant" } }

```

2. **content_block_start:** New content block (text or tool use)

```

{ "type": "content_block_start", "index": 0, "content_block": { "type": "text" } }

```

3. **content_block_delta:** Incremental content

```

{ "type": "content_block_delta", "index": 0, "delta": { "type": "text_delta", "text": "Hel

```

4. **content_block_stop:** End of content block

```

{ "type": "content_block_stop", "index": 0 }

```

5. **message_delta:** Usage and stop reason

```

{
  "type": "message_delta",
  "delta": { "stop_reason": "end_turn" },
  "usage": { "output_tokens": 123 }
}

```

6. **message_stop:** End of message

```
{ "type": "message_stop" }
```

Event Processing: 1. SDK receives SSE chunk from API 2. SDK parses and emits event 3. StreamAdapter handles event 4. StreamAdapter emits Pi-compatible event 5. Runner forwards to Gateway 6. Gateway updates user message progressively

1.2.8 8. Completion Phase

Components: Runner → Logger (optional) → Gateway → User

1. Extract final message with complete text
2. Extract usage statistics (input/output tokens)
3. **Optional:** Log usage to JSONL file

```
{
  "ts": "2026-02-09T20:00:05.000Z",
  "stage": "usage",
  "runId": "run_123",
  "sessionId": "session_456",
  "provider": "anthropic",
  "modelId": "claude-sonnet-4-5-20250929",
  "usage": {
    "input_tokens": 456,
    "output_tokens": 123,
    "cache_creation_input_tokens": 0,
    "cache_read_input_tokens": 0
  }
}
```

4. Process tool calls (if any)
 5. Execute tools and continue conversation (recursive)
 6. Send final response to user
-

1.2.9 9. Error Handling

Components: All layers

1.2.9.1 Error Types:

1.2.9.1.1 A. Authentication Errors (401)

API → SDK → Stream → Runner → Gateway → User

Error: "Invalid API key"

User Message: "Authentication failed. Please check your API key."

If logging enabled:

Log error in usage stage

1.2.9.1.2 B. Rate Limit Errors (429)

API → SDK → Stream → Runner → Gateway → User

Error: "Rate limit exceeded"

User Message: "Rate limit reached. Please try again later."

Retry-After header: 60 seconds

1.2.9.1.3 C. Invalid Request Errors (400)

API → SDK → Stream → Runner → Gateway → User

Error: "Invalid request: [details]"

User Message: "Request failed: [error details]"

Common causes:

- Invalid model ID
- Malformed tool definitions
- Exceeded max tokens

1.2.9.1.4 D. Server Errors (500, 503)

API → SDK → Stream → Runner → Gateway → User

Error: "Service unavailable"

User Message: "Anthropic API is temporarily unavailable. Please try again."

1.3 Key Flow Characteristics

1.3.1 Simplicity vs Azure OpenAI

Aspect	Azure OpenAI	Anthropic
Auth	Complex (Managed Identity, Azure CLI)	Simple (API key or token)
Stream Adapter	Custom native adapter	Default Pi AI adapter
Credential Provider	Azure Identity SDK	Direct API key/token
Token Management	Bearer token provider	Static API key
Complexity	High	Low

1.3.2 Advantages

1. **Simpler architecture:** Standard Pi AI integration
2. **Fewer dependencies:** No Azure SDK dependencies
3. **Easier to maintain:** Less custom code
4. **Optional logging:** Payload logging is opt-in
5. **Multiple auth modes:** Flexible authentication

1.3.3 Trade-offs

1. **No keyless auth:** Requires API key or token (vs Managed Identity)
 2. **Manual token rotation:** Tokens must be manually updated
 3. **Limited enterprise features:** No Azure AD integration
-

1.4 Tool Use Flow

When the model requests tool calls:

1. Model returns `tool_use` content blocks
2. Runner extracts tool definitions and arguments
3. Runner executes tools in sandbox
4. Runner creates `tool_result` messages
5. Runner continues conversation with results
6. Repeat until model returns text (`end_turn`)

Example:

User: "What files are in my project?"
Model: Use bash tool → `ls -la`
Runner: Execute → [file list]
Model: "Here are your files: ..."
User: [final response]

1.5 Performance Optimizations

1.5.1 1. Prompt Caching

Anthropic supports prompt caching for repeated context:

```
{  
  "system": [  
    { "type": "text", "text": "...", "cache_control": { "type": "ephemeral" } }  
  ]  
}
```

1.5.2 2. Streaming

All responses use streaming for progressive updates: - Reduces perceived latency - Allows for real-time feedback - Better user experience

1.5.3 3. Session Caching

Runner caches session managers to avoid re-initialization: - Session files are pre-warmed - Agent configuration is reused - Faster subsequent requests

1.6 Comparison: Full Request Lifecycle

1.6.1 Azure OpenAI (Managed Identity)

User → Gateway → Runner
→ Auth (Managed Identity)
→ Azure Identity SDK (bearer token)
→ Custom Stream Adapter
→ Native Azure OpenAI Client
→ Azure API
→ Response

1.6.2 Anthropic (API Key/Token)

User → Gateway → Runner
→ Auth (API key/token)
→ AuthProfiles (load credential)
→ Default Stream Adapter (streamSimple)
→ Anthropic SDK (via Pi AI)
→ Anthropic API
→ Response (+ optional logging)

Key Difference: Anthropic flow has 2 fewer layers (no Azure Identity, no custom adapter)

1.7 Configuration Examples

1.7.1 Provider Config

```
{
  "models": {
    "providers": {
      "anthropic": {
        "auth": "token",
        "baseUrl": "https://api.anthropic.com",
        "api": "anthropic-messages",
        "defaultModel": "claude-sonnet-4-5-20250929"
      }
    }
  }
}
```

1.7.2 Auth Profile (Token Mode)

```
{
  "profileId": "anthropic:token:default",
  "provider": "anthropic",
  "credential": {
    "type": "token",
    "provider": "anthropic",
  }
}
```

```
    "token": "sk-ant-api03-..."
  }
}
```

1.7.3 Environment Variables

```
# Auth
export ANTHROPIC_API_KEY=sk-ant-api03-...

# Optional: Payload Logging
export CLAWDBOT_ANTHROPIC_PAYLOAD_LOG=true
export CLAWDBOT_ANTHROPIC_PAYLOAD_LOG_FILE=/var/log/anthropic-payload.jsonl
```

1.8 Summary

The Anthropic end-to-end flow is **simpler and more straightforward** than Azure OpenAI:

Advantages: - Standard Pi AI integration (no custom adapter) - Simple authentication (API key or token) - Fewer dependencies - Optional payload logging - Easier to debug and maintain

Limitations: - No keyless authentication (requires API key/token) - No enterprise IAM integration - Manual credential management

Best for: - General-purpose AI integration - Rapid development - Projects without Azure infrastructure - Multi-cloud deployments